



# GPT-5 Prompting Guide

Below is a consolidated, developer-focused guide that extracts and organises all concepts, topics, and key information about prompting GPT-5 from the OpenAI Cookbook page and the Prompt Engineering docs sections relevant to GPT-5. This is structured for quick reference when building agents, coding copilots, and production apps, with practical guidance on roles, structure, tool use, testing, and migration.

Note: To package this into a downloadable PDF, copy this document into a markdown-to-PDF tool or a word processor that exports to PDF.

## Overview

GPT-5 is positioned as a flagship model with substantial gains in agentic task performance, coding, raw intelligence, and steerability, and benefits from adopting canonical tools and prompt optimisation practices derived from real-world usage. The guide emphasises iterating on prompts, using the new API features effectively, and leveraging a prompt optimiser for tuning, while acknowledging that prompting always benefits from experimentation.<sup>[1]</sup>

## Core Prompting Principles

- Treat prompting as an iterative process, with experiments and evals guiding improvements in instruction adherence, planning, and tool use.<sup>[2]</sup> <sup>[1]</sup>
- Prefer concise, explicit instructions and clear structure that define responsibilities, tool use, and output formats; GPT models respond best to precise rules compared with high-level guidance alone.<sup>[2]</sup>
- Use role-based messages or the instructions parameter to assert high-level behaviour, tone, and rules; developer instructions take precedence over user content in execution order.<sup>[2]</sup>
- Organise prompts with headers, lists, and XML-style delimiters to segment identity, rules, examples, and context, improving comprehension and reliability.<sup>[2]</sup>
- Apply few-shot examples to teach patterns; include diverse inputs and desired outputs to steer behaviour predictably.<sup>[2]</sup>

## Prompt Structure and Formatting

- Recommended sections in a developer message:
  - Identity: purpose, communication style, high-level goals.<sup>[2]</sup>
  - Instructions: rules, do's and don'ts, workflow steps, and tool-calling logic; resolve conflicts and ensure specificity.<sup>[2]</sup>

- Examples: input-output pairs that model desired behaviour.<sup>[2]</sup>
- Context: additional data or references relevant to the task, placed near the end for reuse and modularity.<sup>[2]</sup>
- Use Markdown and XML-like tags to mark boundaries, indicate metadata, and separate instructions from context; this supports long-context clarity and reduces confusion.<sup>[2]</sup>
- Place critical instructions and the user query at both the top and bottom of the prompt to improve adherence in long-context settings, which was shown to outperform bottom-only placement and marginally beat top-only placement in experiments.<sup>[2]</sup>
- Choose effective delimiters for large contexts; XML and similar formats tested well, while JSON performed worse for boundary clarity in long-context evals.<sup>[2]</sup>

## Instruction Hierarchy and Roles

- Developer messages define rules and are prioritised over user messages; assistant content is the model's output and does not override developer rules.<sup>[2]</sup>
- The instructions parameter sets high-level behaviour for the current request only; it is not automatically carried across turns unless conversation state handling includes it explicitly via conversation management parameters.<sup>[2]</sup>

## Context and RAG

- Include only relevant context to constrain answers and ground outputs, commonly with retrieval-augmented generation; be mindful of context window limits and prioritise important content.<sup>[2]</sup>
- For long contexts, GPT models like GPT-5 benefit from explicit structure and selective relevance to avoid distraction by irrelevant content.<sup>[2]</sup>

## Few-shot Learning

- Include a small set of representative examples that cover variations of inputs and the exact output style desired, especially when teaching a classification, transformation, or formatting task.<sup>[2]</sup>
- Keep examples consistent with the rules stated in the instructions section to avoid conflicts.<sup>[2]</sup>

## Coding and Engineering Prompts for GPT-5

- Explicit role and workflow guidance:
  - Frame GPT-5 as a software engineering agent with defined responsibilities and boundaries.
  - Specify tools it should use for tasks such as code editing, diffing, testing, or file operations, and when not to use interactive execution modes unless required.<sup>[3]</sup>
- Testing and validation:

- Instruct the model to write and run unit tests or verification commands, and validate patches thoroughly; some automated patch tools can report success even if changes fail, so require verification steps in the prompt.<sup>[3]</sup>
- Tool use examples:
  - Include concrete examples of tool invocation with expected inputs and outputs to standardise behaviour and reduce function-call variance.<sup>[3]</sup>
- Markdown standards:
  - Require clean output formatting with inline code, fenced code blocks, clear lists and tables when appropriate, and consistent formatting of file paths, function names, and classes with backticks for readability and parsing.<sup>[3]</sup>
- Front-end engineering:
  - GPT-5 performs well for front-end scaffolding and contributions to larger codebases; recommended libraries include Tailwind CSS, shadcn/ui, Radix Themes for UI, Lucide or other icon sets, and Motion for animation when guiding the model to pick a stack.<sup>[3]</sup>

## Agentic Behaviour and Planning

- Encourage persistence, tool-calling discipline, and planning with explicit reminders:
  - Persistence: continue until the task is fully resolved before ending the turn.
  - Tool-calling: read files, inspect repos, or call functions when unsure, rather than guessing.
  - Planning: plan before each tool call and reflect on outcomes, avoiding unstructured sequences of tool calls.<sup>[2]</sup>
- Use preambles for tool calls:
  - Instruct GPT-5 to provide a brief rationale or plan immediately before each tool call to increase transparency and improve tool accuracy without excess reasoning overhead.<sup>[3]</sup>
- Chain-of-thought placement:
  - While step-by-step reasoning can help break down tasks, GPT models benefit more from explicit instructions; if using CoT, keep it concise and focused on planning and document selection, then instruct specific outputs.<sup>[2]</sup>

## Reliability, Failure Modes, and Guardrails

- Common pitfalls and mitigations:
  - Over-constraining tool use can cause hallucinated tool arguments or null calls; explicitly instruct the model to ask for missing parameters rather than making assumptions.<sup>[2]</sup>
  - Sample phrases in prompts can cause repetition; instruct variation when presenting examples to avoid verbatim reuse.<sup>[2]</sup>
  - Without strict output rules, models may add unnecessary prose or formatting; specify output schema, verbosity, and exact formats.<sup>[2]</sup>

- Instruction conflicts:
  - If instructions conflict, models tend to follow the instructions closer to the end of the prompt; ensure consistency and order rules deliberately. <sup>[2]</sup>

## Reusable Prompts and Prompt Caching

- Reusable prompts:
  - Maintain prompts in the dashboard with variables for runtime substitution, enabling controlled updates and versioning without changing code. <sup>[2]</sup>
- Prompt caching:
  - Place reusable content early in the prompt and in the first API parameters to maximise cache savings for both cost and latency. <sup>[2]</sup>

## Migration and API Guidance for GPT-5

- Use the Responses API:
  - GPT-5 works best with the Responses API, which supports passing chain of thought between turns to reduce generated reasoning tokens, improve cache hit rates, and lower latency compared with older APIs. <sup>[3]</sup>
- Reasoning effort and verbosity:
  - Control reasoning effort levels and text verbosity explicitly in the Responses API parameters, mapping cleanly from the chat-completions style options when migrating. <sup>[3]</sup>
- Tooling:
  - Define custom tools through the Responses API tools field; provide descriptions and enforce correct usage with clear prompting and examples; consider allowed tool lists and auto mode for routing. <sup>[3]</sup>
- Migration paths:
  - From o3: start with medium reasoning and prompt tuning, increase to high if needed.
  - From gpt-4.1: start with minimal or low reasoning and tune prompts; increase if performance needs rise.
  - From smaller models: gpt-5-mini or gpt-5-nano with prompt tuning are suggested replacements for speed-cost trade-offs. <sup>[3]</sup>
- Prompt optimiser:
  - Use the official prompt optimiser to automatically adapt existing prompts to GPT-5 best practices before manual refinement. <sup>[1] [3]</sup>

## Practical Steps and Checklist

- Define agent identity and responsibilities; include persistence, planning, and disciplined tool-calling reminders. <sup>[1]</sup> <sup>[2]</sup>
- Specify tool interfaces, allowed tool sets, and example invocations; instruct the model to explain its tool choice briefly before calling tools. <sup>[3]</sup>
- Set output standards: format, schemas, verbosity, and markdown rules for code-centric tasks; require validation and tests. <sup>[3]</sup>
- Use clear delimiters and put critical instructions at both top and bottom; include minimal but representative few-shot examples. <sup>[2]</sup>
- Include only necessary context; add RAG or file references when needed and keep them delineated and searchable. <sup>[2]</sup>
- Configure Responses API parameters for reasoning effort and verbosity; enable prompt caching and maintain reusable prompts for version control. <sup>[3]</sup> <sup>[2]</sup>
- Build evals for instruction adherence, tool-call accuracy, and task success; iterate with the prompt optimiser and manual audits. <sup>[1]</sup> <sup>[3]</sup>

## Example Skeleton for a GPT-5 Engineering Agent

- Identity
  - "You are a senior software engineer agent responsible for safe, test-driven changes. You plan before acting, call tools only when you have sufficient parameters, and verify changes with tests." <sup>[3]</sup> <sup>[2]</sup>
- Instructions
  - Planning, persistence, and tool-calling rules.
  - Coding standards, diff format, and markdown conventions.
  - Testing and verification steps after changes.
  - Ask for missing information rather than guessing. <sup>[3]</sup> <sup>[2]</sup>
- Tools
  - Enumerate tools with names, descriptions, and required parameters; include small examples of how to invoke each correctly. <sup>[3]</sup>
- Examples
  - One end-to-end patch example with tests and a verified outcome; one failure-handling example where the agent asks for more info. <sup>[3]</sup> <sup>[2]</sup>
- Context
  - Target repository structure summary, coding standards doc, and key interfaces or APIs as references with clear delimiters. <sup>[2]</sup>

Place a short summary of critical rules again at the end of the prompt to reinforce adherence in long contexts. <sup>[2]</sup>

## Notes on Model Behaviour

- GPT models are highly steerable with explicit instructions; reasoning models are more tolerant of high-level goals with fewer specifics; tailor prompting depth accordingly.<sup>[2]</sup>
- For long-context use, delimiter choice, instruction placement, and selective relevance materially impact performance.<sup>[2]</sup>
- GPT-5-specific improvements show up strongly in coding, frontend, and agentic tool use when the prompt enforces structure, tests, and disciplined tool invocation.<sup>[1] [3]</sup>

## References

- OpenAI Cookbook: GPT-5 prompting guide.<sup>[1]</sup>
- OpenAI Docs: Prompt engineering, roles, structure, few-shot, long-context strategies, and agent reminders; migration and Responses API specifics including reasoning effort, verbosity, tools, preambles, and GPT-5 prompting notes.<sup>[3] [2]</sup>

✱

1. <https://platform.openai.com/docs/guides/prompt-engineering>
2. [https://cookbook.openai.com/examples/gpt-5/gpt-5\\_prompting\\_guide](https://cookbook.openai.com/examples/gpt-5/gpt-5_prompting_guide)
3. <https://platform.openai.com/docs/guides/latest-model>